Команда Сбера

Протоколы взаимодействия AI-агентов

Тращенков Сергей

Разработка и внедрение AI-агентов · МФТИ

Материалы

Telegram-канал

Какой протокол для какой задачи

Задача примера — найти поезд и подобрать продукты в дорогу.

MCP

Вызов инструментов
Чтение ресурсов
Получение промптов

A2A

Отправка задачи
другому агенту
и получение результата

AG-UI / A2UI

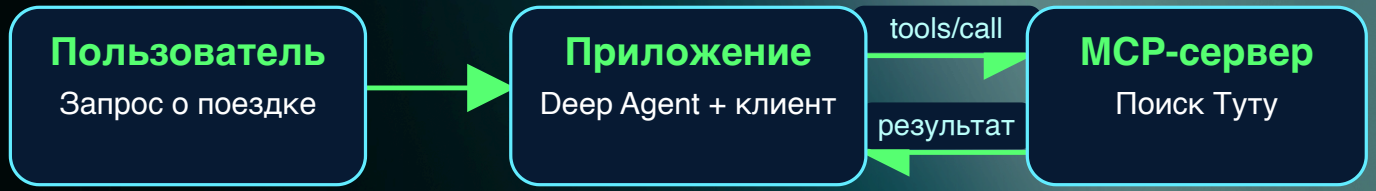
AG-UI: события агента

A2UI: компоненты интерфейса

ACP передаёт запросы редактора агенту. Навык (Skill) содержит инструкции и вспомогательные файлы.

MCP: приложение, клиент и сервер

Приложение (host) создаёт MCP-клиента и передаёт модели описания инструментов.
Клиент отправляет вызов серверу, сервер выполняет операцию.



Материалы

Telegram-канал

MCP: инструменты, ресурсы и промпты

Tools

Вызываемые операции с аргументами и результатом.

`search_rail`

Resources

Данные, которые клиент читает по URI.

`course://overview`

Prompts

Шаблоны сообщений с параметрами.

`prepare_lecture_brief`

> В примере клиент получает все три: список инструментов, текст ресурса и сообщения из промпта.

MCP: как вызывается инструмент

Модель

Выбирает инструмент и задаёт аргументы

→

Приложение

Отправляет MCP-запрос

→

Сервер

Выполняет операцию

В LangChain результат записывается в ToolMessage. Модель получает его при следующем вызове.

MCP: чтение ресурса по URI

Resource

Описание курса по URI

→

Host

Читает содержимое

→

Запрос к модели

Содержит текст ресурса

Клиент читает описание курса по URI course://overview. Приложение добавляет этот текст в запрос к модели.

MCP: получение шаблона промпта

Аргументы

```
topic_name="mcp"
```

```
audience="студенты"
```

Результат сервера

Сообщения с темой справки, аудиторией и требованиями к ответу

Сервер подставляет аргументы в шаблон и возвращает сообщения. Приложение передаёт их модели.

MCP: ресурс и промпт в запросе к модели

Приложение передаёт модели описание курса, промпт справки и описания инструментов чтения файлов.



MCP: версии спецификации и Python SDK

Спецификация

`2025-11-25`
↓
`2026-07-28`

Python SDK

`v1` → `v2`

Версия Python-библиотеки

Спецификация задаёт формат обмена. Версия SDK определяет Python API для этого обмена.

MCP: начало обмена до и после обновления

2025-11-25	2026-07-28
initialize и согласование возможностей	Версия и возможности в _meta запроса
HTTP-сессия Mcp-Session-Id	Явные идентификаторы объектов в аргументах
Сведения при инициализации	server/discover и метаданные запроса

В блокноте отправляем tools/call без initialize, затем проверяем подключение клиента предыдущей редакции.

MCP: запрос дополнительного ввода (MRTR)

Сервер

Возвращает input_required

→

Приложение

Получает ввод пользователя

→

Повтор запроса

Передаёт inputResponses

Подписки: subscriptions/listen. Tasks вынесены в расширение. Roots, Sampling и Logging помечены устаревающими.

Материалы Telegram-канал

MCP Python SDK: переход с v1 на v2

Сервер

`FastMCP` → `MCPServer`

Параметры транспорта передаются в run()

Клиент

`from mcp import Client`

async with сохраняет соединение до выхода из блока

На сервере используем декораторы для tools, resources и prompts; на клиенте — async with Client(...).

Материалы

Telegram-канал

MCP в LangChain: новый адаптер

Прежнее подключение	Используемое подключение
langchain-mcp-adapters	langchain[mcp]
MultiServerMCPClient	MCPAdapter
get_tools()	await adapter.list_tools()

MCPAdapter подключаем через async with. Ресурсы и промпты читаем клиентом MCP SDK. Модуль langchain.mcp имеет статус beta.

Материалы Telegram-канал

MCP Туту: список доступных инструментов

Подключение

URL MCP-сервера Туту

→

Список инструментов

list_tools()

→

Проверка

Имена, описания, схемы

Вызываем list_tools() и смотрим имена, описания и схемы аргументов до запуска агента.

Материалы Telegram-канал

MCP Туту: параметры инструмента в JSON Schema

Описание аргументов

Схема задаёт имена и типы аргументов. Список required указывает обязательные поля.

Допустимое значение null

```
anyOf: [integer, null]
```

API модели должен принимать такую схему

GigaChat API v2 принимает используемые схемы с nullable-полями. Передаём схемы, полученные от MCP-сервера.

Материалы Telegram-канал

MCP Туту: подключение к Deep Agent

```
tools = await adapter.list_tools()
agent = create_deep_agent(model=model, tools=tools)
result = await agent.ainvoke({"messages": messages})
```

Передаём инструменты из MCPAdapter в create_deep_agent. Модель выбирает инструмент и задаёт аргументы вызова.

MCP Туту: вызов и результат в истории

AIMessage

search_rail и аргументы

→

ToolMessage

Ответ Туту

→

AIMessage

Поезд, время и цена

Сопоставляем ответ агента с ToolMessage: дату, направление, отсутствие пересадок, время и цену.

MCP: сервер для чтения учебных материалов

Resources

Описание курса
Сведения о темах

Prompt

Справка по теме
Заданная аудитория

Tools

Список файлов
Чтение начала файла

Сервер читает файлы из data/course и предоставляет tools, resources и prompts для подготовки справки.

Материалы

Telegram-канал

MCP по stdio: запуск сервера из клиента

MCP-клиент

Запускает Python

→

stdin / stdout

Сообщения JSON-RPC

→

Сервер

Читает учебные файлы

Вход в async with Client(...) запускает сервер. Выход из блока закрывает соединение и завершает процесс.

Материалы Telegram-канал

MCP: агент готовит справку по теме

До запуска

Resource + prompt

→

Во время запуска

Список файлов → чтение

→

После запуска

Справка и история

В блокноте выводим сообщения перед вызовом модели, вызовы инструментов и готовую справку.

Материалы

Telegram-канал

MCP по HTTP: запрос и ответ

Запускаем сервер учебных материалов по Streamable HTTP. В блокноте смотрим отправленный запрос и ответ.

Запрос

Метод tools/call
Аргументы операции
Версия и возможности в _meta

Ответ

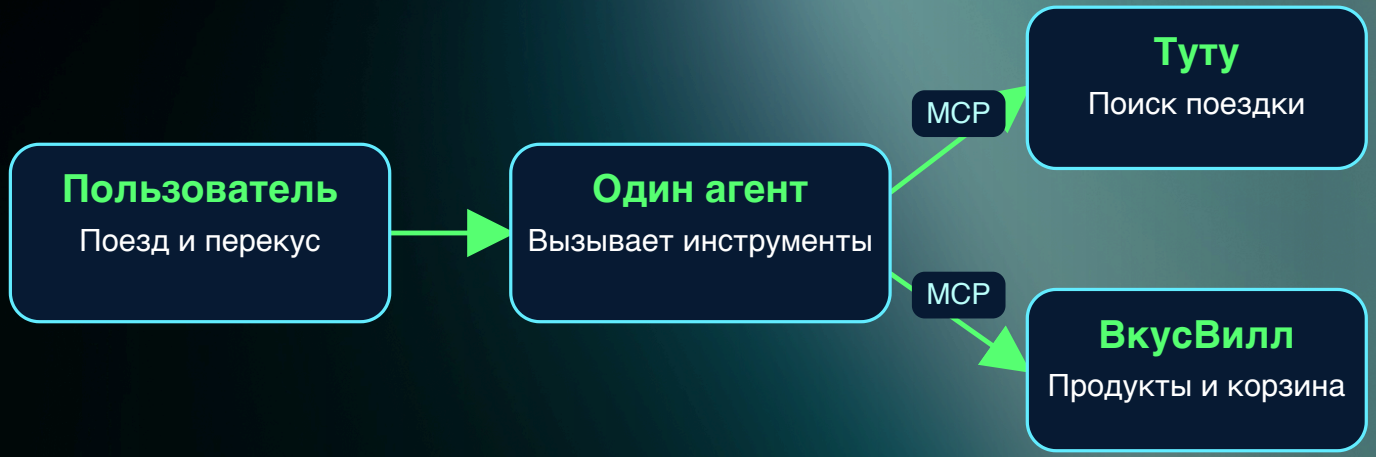
Результат операции
resultType
Метаданные сервера

Streamable HTTP может передавать SSE. Отдельный старый транспорт HTTP+SSE помечен устаревающим.

Материалы

Telegram-канал

MCP: один агент с Туту и ВкусВиллом



Один Deep Agent получает инструменты обоих MCP-серверов и выбирает вызовы для поиска поезда и продуктов.

MCP ВкусВилла: от поиска к корзине

Поиск

Названия и id продуктов

→

Выбор

3–4 позиции перекуса

→

Корзина

products: id + количество

Результат — ссылка на корзину с выбранными продуктами. Оформление и оплату оставляем пользователю.

MCP и OAuth: доступ к личным данным

Приложение

Начинает авторизацию с PKCE

→

Пользователь

Подтверждает доступ

→

Сервис

Выдаёт токен доступа

Личный кабинет требует OAuth-авторизации пользователя. В примерах вызываем публичные операции Туту и ВкусВилла.

Практика MCP: добавить инструмент на сервер

Добавить

Инструмент подсчёта учебных файлов по расширению

Проверить

list_tools → ручной вызов → вызов агентом

Доступ к файлам остаётся ограничен папкой data/course.

Команда Сбера

A2A, AG-UI, A2UI и ACP

Задачи между агентами, события для приложения и работа с редактором · 45 минут

Материалы

Telegram-канал

A2A: координатор и агент поездок



Координатор отправляет задачу по A2A. Агент поездок вызывает MCP Туту и возвращает результат; координатор подбирает продукты через MCP ВкусВилла.

A2A: адрес и возможности агента в Agent Card

Описание агента

Имя и назначение
Версия реализации

Интерфейс

URL
JSONRPC,
protocolVersion

Возможности

Skills
Форматы сообщений
Capabilities

Поле skills перечисляет задачи агента. Формат SKILL.md с инструкциями разбираем в дополнительном разделе.

A2A: сообщение, задача и результат

SendMessage

Запрос пользователя

→

Task

Статус выполнения

→

Artifacts

Полученные результаты

В ответе проверяем статус completed и читаем текст поездки из artifacts.

A2A: два сервера с одним агентом

Общая логика

Deep Agents + GigaChat
Один системный промпт
Одни инструменты Туту по MCP

Две реализации

Сервер на a2a-sdk

LangGraph Agent Server

Оба сервера запускают агента из agents/travel.py. Отправляем им одинаковый запрос одним клиентом.

A2A, способ A: сервер на a2a-sdk

Agent Card

Описываем интерфейс

→

AgentExecutor

Запускаем общий граф

→

TaskUpdater

Статус и результат

AgentExecutor получает A2A-запрос, вызывает агента и через TaskUpdater возвращает статус и результат.

A2A, способ B: LangGraph Agent Server

```
from agents.travel import travel_graph

graph = travel_graph
```

```
{"graphs": {"travel": "./servers/agent_server.py:graph"}}
```

> langgraph dev запускает сервер с A2A-адресом для графа travel. Соединение агента с MCP Туту открыто во время выполнения.

A2A: сравнение двух серверов

Проверяем	Оба варианта
Создание агента	Одна и та же функция
Источник данных	MCP Туту
Вход	Одинаковый запрос о поездке
Выход	Завершённая задача и текст в artifacts

Для обоих ответов проверяем дату, направление и соответствие данным Туту. Цена и формулировки могут меняться между запросами.

A2A: совместимость клиента и сервера

Наблюдение

Клиент a2a-sdk с protobuf-моделями отклоняет дополнительное поле kind в ответе Agent Server

Учебный клиент

JSON-RPC через HTTP
Статус задачи и текст из artifacts

Поэтому в блокноте используем JSON-RPC-клиент, который принимает ответы обоих серверов.

A2A и MCP: поездка и продукты в одном запросе

travel_agent

A2A-запрос о поездке

→

MCP ВкусВилла

Поиск и ссылка на корзину

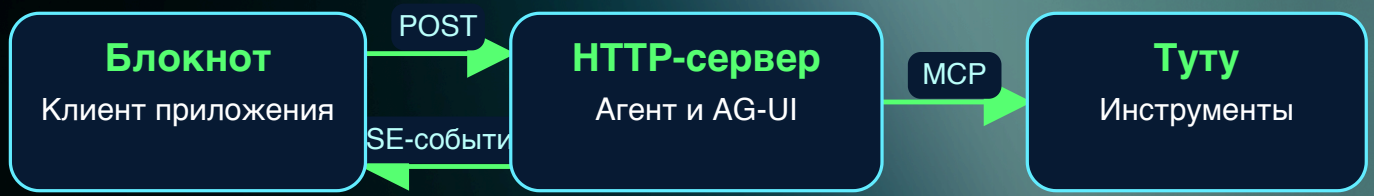
→

Ответ

Поезд и продукты

Повторяем запрос с адресом второго A2A-сервера. Координатор по-прежнему получает поездку и ссылку на корзину.

AG-UI: события для приложения

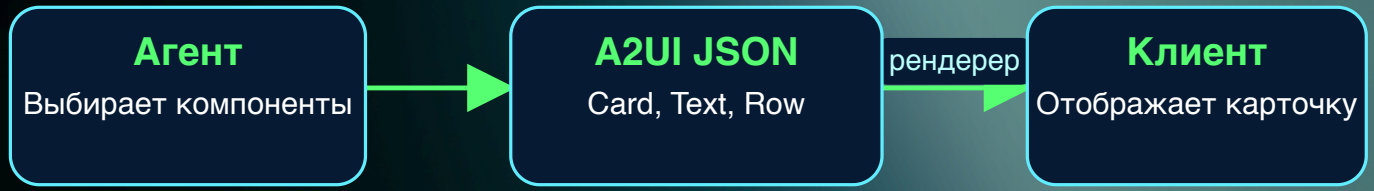


Ячейка блокнота запускает HTTP-сервер отдельным процессом, отправляет запрос и читает SSE-события.

AG-UI: как клиент отображает события

Событие	Действие клиента
RUN_STARTED	Индикатор работы
TOOL_CALL_START / TOOL_CALL_RESULT	Название инструмента и его результат
TEXT_MESSAGE_CONTENT	Добавление фрагмента ответа
RUN_FINISHED / RUN_ERROR	Завершение или сообщение об ошибке

A2UI: описание интерфейса



Агент возвращает JSON с компонентами из каталога. Клиент строит по нему интерфейс.

A2UI: от JSON к карточке поезда

Структура

createSurface
updateComponents
root → Card → Text

Данные

Номер поезда
Отправление и прибытие
Цена

Используем A2UI v0.9. В блокноте выводим JSON и строим из него статическую карточку поезда.

ACP: редактор и агентный процесс



Ячейка блокнота запускает ACP-сервер и обменивается с ним JSON-RPC через stdin и stdout.

ACP: создание сессии и отправка запроса

initialize

Версия и возможности

→

session/new

Рабочий каталог

→

session/prompt

Запрос пользователя

Агент присылает session/update и завершает ответ с stopReason=end_turn. JSON-RPC идёт в stdout, логи — в stderr.

Материалы Telegram-канал

Покупка через агента: участники и согласие

Пользователь

Выбирает товар
и разрешает покупку

→

Агент и продавец

Уточняют покупку

→

Платёжный сервис

Проверяет
разрешение

Обзор протоколов: как передать заказ и подтвердить согласие на оплату. В блокноте платежей нет.

Протоколы покупки и оплаты

Протокол	Назначение
Agentic Commerce Protocol	Данные оформления заказа между агентом и продавцом
UCP	Поиск, корзина, оформление, работа с заказом
AP2	Подтверждения полномочий агента и согласия на оплату
x402	Оплата доступа к HTTP-ресурсу

Agent Client Protocol и Agentic Commerce Protocol имеют одинаковую аббревиатуру ACP.

Контрольные вопросы

Инструменты и контекст

Кто читает ресурс?
Что возвращает промпт?
Где выполняется инструмент?

Агенты и приложение

Кто выполняет поиск по A2A?
Кто отображает компоненты A2UI?
Что передаёт stdout ACP?

Материалы

Telegram-канал

Команда Сбера

Skills: дополнительный материал

Пример навыка: создать QR-код, прочитать его и проверить содержимое

Skills: содержимое папки навыка

SKILL.md

Имя и описание
Процедура выполнения
Требования

Файлы

qr_generate.py
qr_read.py
Автор и версия навыка

По имени и описанию агент выбирает навык, читает SKILL.md и выполняет указанные шаги.

Skills: создание и чтение QR-кода



Агент создаёт QR-код и читает его обратно. В отдельной ячейке проверяем, что декодированная строка совпадает с исходной.

Skills: поиск и загрузка в LangChain

list_skills

Краткие описания

→

load_skill

Полный SKILL.md

→

Агент

План выполнения

В этом примере агент находит навык через list_skills и читает SKILL.md через load_skill. Запуск скриптов показан в предыдущем примере с Deep Agents.

Skills: установка QR-библиотек на разных ОС

ОС	Нативная библиотека
Windows x64	DLL в pyzbar; при ошибке — Visual C++ Runtime 2013 x64
macOS	brew install zbar
Debian/Ubuntu	apt-get install libzbar0

После uv sync запускаем check_env.py --qr: он создаёт и читает QR-код, проверяя доступность zbar.

Спецификации и документация

MCP и инструменты

- Спецификация MCP
- MCP Python SDK v2
- MCP в LangChain

Агенты и интерфейсы

- Agent2Agent
- AG-UI
- A2UI
- Agent Client Protocol

Материалы

Telegram-канал

Документация протоколов покупки и оплаты

- Agentic Commerce Protocol
- Universal Commerce Protocol
- Agent Payments Protocol
- x402

> Ссылки на документацию также приведены в блокноте.

Материалы Telegram-канал